
TCPDUMP

Complete Field Guide for EKS & Kubernetes

From Zero to Production Debugging

- Network Packet Analysis
- Pod-to-Pod Traffic
- Performance Impact
- DNS Debugging
- Request Body Filtering
- Historical Analysis
- Latency Measurement
- EKS Node Capture
- Production Safety

Based on real EKS debugging sessions & best practices

Table of Contents

1	What is Tcpdump?	Network packet capture fundamentals
2	Core Concepts & Terminology	Essential vocabulary for understanding output
3	Essential Flags Reference	Every important flag explained
4	The -nn Flag Deep Dive	Why this flag matters most in EKS
5	Basic Filters with Real Outputs	Host, port, protocol and combined filters
6	TCP Flags — Reading the Handshake	SYN, ACK, RST and what they mean
7	DNS — The Hidden Killer in EKS	How DNS failures affect every request
8	Using Tcpdump in EKS	Node, debug pod and nsenter methods
9	How Debug Pod Sees Pod Traffic	hostNetwork and nsenter explained
10	Measuring Latency	Network RTT, app latency, DNS latency
11	Filtering by Request ID / Body	Targeting specific requests in traffic
12	-A vs -s 0 Difference	Capture vs display flags explained
13	Can You Rewind Traffic?	Rolling captures and historical alternatives
14	Performance Impact on Pods	CPU/memory cost and safety rules
15	Production Safety Rules	What never to do in production
16	Quick Reference Cheatsheet	Copy-paste ready commands
17	Troubleshooting Decision Tree	Step-by-step diagnostic flow

Chapter 1 — What is Tcpdump?

Think of your network like a **highway**. Data packets are the **cars** driving through it. **Tcpdump is a traffic camera** — it captures and shows you every packet passing through your network interface in real time, without interrupting or slowing down the traffic itself.

It is a command-line packet analyzer built into Linux that lets you:

- See what traffic is going in/out of your system in real time
- Debug network issues between microservices
- Verify if packets are actually reaching their destination
- Measure latency between services at the network layer
- Inspect HTTP request and response bodies
- Detect connection resets, timeouts, and routing failures
- Capture traffic to files for later analysis in Wireshark

What Tcpdump IS vs IS NOT

Tcpdump IS	Tcpdump IS NOT
A passive packet observer (copy of traffic)	A proxy or middleman in the traffic path
A real-time capture tool	A historical log viewer (by default)
A network-layer debugger (L3/L4)	An app-layer APM or tracing tool
Free and built into Linux	A replacement for proper observability stack
Safe — never adds latency to requests	Useful on bastion hosts for pod-to-pod traffic

■ **Key insight:** Tcpdump works by receiving a *kernel copy* of each packet. Your actual packets continue their journey unaffected. This means tcpdump can NEVER add latency to your requests — it is always a silent observer.

Chapter 2 — Core Concepts & Terminology

Term	What it Means	Example in EKS
Interface	The network card tcpdump listens on	eth0, any, lo
Packet	A small chunk of data traveling over the network	One HTTP request split into chunks
BPF Filter	Berkeley Packet Filter — language to select packets	port 8080 and host 10.0.1.10
Port	A numbered door on a host	80=HTTP, 443=HTTPS, 53=DNS, 5432=Postgres
Snapshot Length (-s)	How many bytes of each packet to capture	-s 0 means capture ALL bytes
RTT	Round Trip Time — packet travel time there and back	SYN to SYN-ACK time
Network Namespace	Isolated network environment each pod lives in	Each pod has its own eth0
veth	Virtual ethernet cable connecting pod to node bridge	veth1a2b3c on the node
CNI	Container Network Interface — manages pod networking	aws-vpc-cni, Calico, Flannel
CoreDNS	The DNS server inside Kubernetes (IP: 10.96.0.10)	Resolves service names to IPs
mTLS	Mutual TLS — encrypted traffic in service meshes	Istio/Envoy sidecar traffic
NXDOMAIN	DNS response: domain/service name does not exist	Typo in service name

How Packets Flow in EKS

Understanding traffic flow is critical to knowing WHERE to run tcpdump. Every pod-to-pod packet travels through the node's network interfaces:

```

Client Request
  |
  v
Load Balancer (ALB/NLB)
  |
  v
EKS Node eth0 (10.0.1.5)      <-- tcpdump here = see ALL pod traffic
  |
  v
Node Bridge (aws-cni / cbr0)
  |           |
  v           v
veth1 (Pod A)   veth2 (Pod B) <-- tcpdump here = see specific pod
  |           |
  v           v
Pod A eth0      Pod B eth0      <-- nsenter here = pod's own view
(10.0.1.10)     (10.0.4.22)

```

Chapter 3 — Essential Flags Reference

Flag	What it Does	When to Use
-i eth0	Listen on a specific interface	When you know which NIC to watch
-i any	Listen on ALL interfaces simultaneously	Most common in EKS debugging
-n	Don't resolve hostnames to DNS names	Speeds up capture slightly
-nn	Don't resolve hostnames OR port names (e.g. 80 stays 80, not 'http')	ALWAYS use in EKS
-v	Verbose — more packet detail (TTL, checksum, etc.)	General network debugging
-vvv	Very verbose — maximum protocol detail	Deep protocol inspection
-A	Show packet content as human-readable ASCII text	Reading HTTP request/response bodies
-X	Show packet in HEX + ASCII side by side	Binary protocol debugging
-s 0	Capture FULL packet — no size limit (default is only 96 bytes)	Required to see HTTP body
-w file.pcap	Write raw capture to binary file	Production captures, Wireshark analysis
-r file.pcap	Read and analyze from a pcap file	Replaying past captures
-c 100	Stop after capturing N packets	Safety limit in production
-G 3600	Rotate output file every N seconds	Rolling capture windows (24hr history)
-W 24	Keep only last N rotated files	Disk space management
-tt	Print precise Unix timestamp (e.g. 1714650181.100123)	Latency calculation with awk
--line-buffered	(When piping to grep) Flush output immediately	Required for real-time grep filtering

■ ■ **Most important combo in EKS:** Always start with **tcpdump -i any -nn** as your base. Add flags from there depending on what you need to see.

Chapter 4 — The -nn Flag Deep Dive

The `-nn` flag is one of the most important flags to understand in EKS environments. Without it, `tcpdump` pauses on every single packet to perform DNS lookups — both for IP addresses and port numbers — creating a cascade of problems.

Without -nn (Bad — Slow & Noisy)

```
tcpdump -i eth0 port 80
```

Output:

```
14:23:01.123456 IP ip-10-0-1-45.us-east-1.compute.internal.http >
ip-10-0-2-30.us-east-1.compute.internal.webcache: Flags [S], seq 123456
```

Problems this causes:

- ✗ Pauses to resolve EVERY IP address to a hostname — slows capture output
- ✗ Pauses to resolve EVERY port number to a service name (80 → 'http', 443 → 'https')
- ✗ Generates extra DNS traffic — this POLLUTES your own capture with noise
- ✗ Long hostname strings are extremely hard to read
- ✗ On high-traffic nodes, DNS resolution can cause output to fall behind

With -nn (Good — Fast & Clean)

```
tcpdump -i eth0 -nn port 80
```

Output:

```
14:23:01.123456 IP 10.0.1.45.80 > 10.0.2.30.54321: Flags [S], seq 123456
```

Raw IPs, raw port numbers. Clean, fast, no side effects.

Flag	Stops Resolving	Example Output
(no flag)	Nothing — resolves everything	compute.internal.http
-n	Hostnames only	10.0.1.45.http
-nn	Hostnames AND port names	10.0.1.45.80 ■ USE THIS

■ **Rule:** Always use `-nn` in EKS and Kubernetes environments. It speeds up capture, reduces noise, and makes output readable. There is no reason not to use it.

Chapter 5 — Basic Filters with Real Outputs

5.1 Filter by Host

```
tcpdump host 10.0.1.10      # traffic TO or FROM this IP
tcpdump src 10.0.1.10      # traffic only FROM this IP (outbound)
tcpdump dst 10.0.1.20      # traffic only TO this IP (inbound)
```

Output — tcpdump src 10.0.1.10 (everything one pod is doing):

```
14:23:01.100 IP 10.0.1.10.54231 > 10.0.4.22.8080:  Flags [P.]  # service call
14:23:01.200 IP 10.0.1.10.54231 > 10.96.0.10.53:   A? orders-svc...  # DNS
14:23:01.300 IP 10.0.1.10.54231 > 10.0.5.11.5432:  Flags [P.]  # DB query
```

In one command you can see everything a pod is doing — service calls, DNS queries, database connections. Incredibly useful for understanding pod behavior.

5.2 Filter by Port

```
tcpdump port 443            # HTTPS traffic
tcpdump port 53             # DNS traffic
tcpdump port 5432           # PostgreSQL
tcpdump portrange 8000-9000 # any microservice in this range
```

5.3 Filter by Protocol

```
tcpdump tcp      # TCP traffic only
tcpdump udp      # UDP traffic only (DNS uses UDP)
tcpdump icmp     # Ping traffic only
```

Healthy ping output (pod is reachable):

```
14:23:01.100 IP 10.0.1.10 > 10.0.4.22: ICMP echo request,  seq 1
14:23:01.101 IP 10.0.4.22 > 10.0.1.10: ICMP echo reply,    seq 1  # <1ms
```

Dead pod or NetworkPolicy blocking — no reply:

```
14:23:01.100 IP 10.0.1.10 > 10.0.4.22: ICMP echo request, seq 1
14:23:02.100 IP 10.0.1.10 > 10.0.4.22: ICMP echo request, seq 2  # retry
14:23:03.100 IP 10.0.1.10 > 10.0.4.22: ICMP echo request, seq 3  # retry
```

5.4 Combining Filters — AND / OR / NOT


```
tcpdump host 10.0.1.10 and port 80
tcpdump src 10.0.1.10 and not port 22
tcpdump \{(port 80 or port 443\) and host 10.0.1.10
tcpdump src 10.0.1.10 and dst port 5432 # Is pod talking to DB?
tcpdump host 10.0.1.10 and host 10.0.4.22 # Traffic between 2 specific pods
```

5.5 Reading HTTP Traffic

```
# Show full HTTP request and response bodies
tcpdump -i any -nn -A -s 0 port 8080

# Grep for specific HTTP paths, methods and status codes
tcpdump -i any -nn -A -s 0 'tcp port 80' | grep -E 'GET|POST|HTTP|Host:'
```

Full HTTP output with -A -s 0:

```
14:23:01.100 IP 10.0.2.30 > 10.0.1.45.8080: Flags [P.], length 120
POST /api/orders HTTP/1.1
Host: orders-service:8080
Content-Type: application/json

{"requestId": "abc123", "userId": "u-999", "amount": 250.00}
14:23:01.145 IP 10.0.1.45 > 10.0.2.30: Flags [P.], length 540
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "success", "orderId": "ord-456"}
```

Chapter 6 — TCP Flags: Reading the Handshake

Every TCP packet carries flags in its header. These flags tell you the STATE of a connection — whether it's opening, transferring data, closing gracefully, or being forcefully killed. Reading these flags is fundamental to understanding what's happening on your network.

Flag	Symbol	Meaning	When You See It
SYN	[S]	Synchronize — starting a connection	Client wants to connect to server
SYN-ACK	[S.]	Server acknowledges and agrees	Server accepts the connection
ACK	[.]	Acknowledgement — received OK	After every packet to confirm receipt
PSH	[P.]	Push — data is being sent now	Actual request/response data
RST	[R.]	Reset — hard connection kill	Port closed, firewall blocking, app crash
FIN	[F.]	Finish — graceful close	Normal connection teardown

Healthy TCP 3-Way Handshake

```
Client                                Server
|                                    |
|----- SYN [S] ----->| 'Can I connect?'
|<----- SYN-ACK [S.] -----| 'Yes, go ahead'
|----- ACK [.] ----->| 'Thanks, connected!'
|----- DATA [P.] ----->| 'Here is my HTTP request'
|<----- DATA [P.] -----| 'Here is my HTTP response'
|----- FIN [F.] ----->| 'I am done, closing'
```

```
14:23:01.100 IP 10.0.2.30.54312 > 10.0.1.45.8080: Flags [S]    # SYN
14:23:01.101 IP 10.0.1.45.8080  > 10.0.2.30.54312: Flags [S.]  # SYN-ACK
14:23:01.102 IP 10.0.2.30.54312 > 10.0.1.45.8080:  Flags [.]   # ACK
14:23:01.200 IP 10.0.2.30.54312 > 10.0.1.45.8080:  Flags [P.]   # Request data
14:23:01.210 IP 10.0.1.45.8080  > 10.0.2.30.54312:  Flags [P.]   # Response data
```

Unhealthy Pattern 1 — SYN with No Reply (Server Unreachable)

```
14:23:01.100 IP 10.0.2.30 > 10.0.1.45.8080: Flags [S]
14:23:02.100 IP 10.0.2.30 > 10.0.1.45.8080: Flags [S] # retry 1
14:23:04.100 IP 10.0.2.30 > 10.0.1.45.8080: Flags [S] # retry 2
# Client keeps retrying – server never responds
```

Causes: Pod is down, wrong port number, SecurityGroup blocking, NetworkPolicy blocking

Unhealthy Pattern 2 — RST (Connection Actively Rejected)

```
14:23:01.100 IP 10.0.2.30 > 10.0.1.45.8080: Flags [S]    # SYN
14:23:01.101 IP 10.0.1.45 > 10.0.2.30:      Flags [R.]  # RESET!
# Server immediately rejects – port closed or firewall
```

Causes: Nothing listening on that port, firewall actively rejecting, app crashed

SYN and RST Capture Commands

```
# Capture only new connection attempts (SYN packets)
tcpdump 'tcp[tcpflags] & tcp-syn != 0'

# Capture only connection resets (RST packets)
tcpdump 'tcp[tcpflags] & tcp-rst != 0'

# Watch RSTs on a specific service
tcpdump 'tcp[tcpflags] & tcp-rst != 0' and port 8080
```

■ **Alert:** If you see RST packets flooding in every 100ms or faster, something is seriously wrong. Check if the pod was OOMKilled, if a load balancer is timing out connections, or if a SecurityGroup rule was changed.

Chapter 7 — DNS: The Hidden Killer in EKS

DNS is the phone book of Kubernetes. Every time a pod wants to talk to another service by name (like `payments-service.default.svc.cluster.local`), it fires a DNS query to **CoreDNS** first. If DNS is slow or broken, *every single request* in your entire cluster is affected — even if your application code and services are perfectly healthy.

How DNS Works in Kubernetes

```
Your Pod (10.0.1.10)
|
|--- DNS Query: 'Where is payments-service?' ---> CoreDNS (10.96.0.10:53)
|
|<-- DNS Reply: 'It is at 10.0.4.22' <----- CoreDNS
|
+--- TCP Connect to 10.0.4.22:8080

Port 53 = DNS. Every service name lookup goes through here FIRST.
```

DNS Debug Command

```
tcpdump -i any -nn port 53
```

Healthy DNS — Query + Quick Reply:

```
14:23:01.100 IP 10.0.1.10.45231 > 10.96.0.10.53: A? payments-service.default.svc.cluster.local
14:23:01.102 IP 10.96.0.10.53 > 10.0.1.10: A payments-service... 10.0.4.22
# DNS latency: 2ms <--- ideal
```

DNS Timeout — CoreDNS Down or Overloaded:

```
14:23:01.100 IP 10.0.1.10 > 10.96.0.10.53: A? payments-service.default.svc.cluster.local
14:23:04.100 IP 10.0.1.10 > 10.96.0.10.53: A? payments-service... # retry after 3s
14:23:07.100 IP 10.0.1.10 > 10.96.0.10.53: A? payments-service... # retry again
# No reply! App will show 'unknown host' or 'connection refused'
```

NXDOMAIN — Service Name Typo:

```
14:23:01.100 IP 10.0.1.10 > 10.96.0.10.53: A? paymens-service.default.svc.cluster.local
14:23:01.102 IP 10.96.0.10 > 10.0.1.10: NXDOMAIN
# 'paymens' not 'payments' -- typo in your code or env var!
```

DNS Latency Spike — CoreDNS Overloaded:

```
14:23:01.100 IP 10.0.1.10 > 10.96.0.10.53: A? orders-svc...
14:23:01.850 IP 10.96.0.10 > 10.0.1.10: A orders-svc... 10.0.4.22
# 750ms DNS latency! Every request is paying this overhead.
```

750ms DNS latency means EVERY single request to any service takes at least 750ms before the actual connection even starts. This is invisible in app logs — only tcpdump reveals it.

Total Latency Formula

```
Total Request Latency = DNS latency
                        + Network RTT (SYN to SYN-ACK)
                        + App processing time
                        + Response transfer time

# Example breakdown:
DNS:      750ms  <-- CoreDNS overloaded (hidden!)
Network:   2ms   <-- healthy
App:       45ms  <-- healthy
Transfer:  5ms   <-- healthy
-----
Total:    802ms  <-- looks like app is slow, but really DNS is the problem
```

CoreDNS Health Check Commands

```
# Check CoreDNS pod status
kubectl get pods -n kube-system | grep coredns

# Check CoreDNS logs for errors
kubectl logs -n kube-system -l k8s-app=kube-dns

# Check CoreDNS resource usage (is it CPU-throttled?)
kubectl top pods -n kube-system | grep coredns

# Test DNS resolution from a debug pod
kubectl run dns-test --image=nicolaka/netshoot -it --rm -- dig payments-service.default.
svc.cluster.local
```

Chapter 8 — Using Tcpdump in EKS

Why Bastion Host Does NOT Work

A very common mistake is trying to run tcpdump on a bastion host to debug pod-to-pod traffic. This does not work because tcpdump only captures traffic that physically flows through the machine it is running on. Pod-to-pod traffic never touches the bastion host.

```
YOUR VPC
+-----+
|                               |
|  Bastion Host (10.0.0.5)      | EKS Cluster | | |
|                               | +-----+ |
|                               | | Pod A 10.0.1.10 | |
|                               | |             | |
|                               | | Pod B 10.0.4.22 | |
|                               | +-----+ |
|                               |      ^      |
|      +-- Cannot see this -----+ |
|      traffic from here!          |
+-----+
```

What Bastion IS Useful For

```
# Use bastion to run kubectl commands that deploy debug pods
kubectl apply -f debug-pod.yaml

# Use bastion to SSM into the right EKS node
aws ssm start-session --target <eks-node-instance-id>

# Use bastion to copy pcap files out for local Wireshark analysis
kubectl cp debug-pod:/tmp/capture.pcap ./capture.pcap
```

Option A — Debug Pod with hostNetwork (Best Practice)

```
# debug-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: net-debug
  namespace: default
spec:
  hostNetwork: true      # Share node network namespace
  hostPID: true          # Access node processes
  nodeName: <your-node>  # Pin to same node as your pods
  containers:
  - name: netshoot
    image: nicolaka/netshoot
    command: ["sleep", "infinity"]
    securityContext:
      privileged: true
  tolerations:
  - operator: Exists
```

```
kubectl apply -f debug-pod.yaml
kubectl exec -it net-debug -- tcpdump -i any -nn port 8080
```

Option B — One-liner Debug Pod

```
kubectl run debug \
  --image=nicolaka/netshoot \
  --overrides='{
    "spec": {
      "hostNetwork": true,
      "nodeName": "ip-10-0-1-5.ec2.internal"
    }
  }' -it --rm -- bash
```

Option C — SSM Directly into EKS Node

```
# Get the instance ID of the node your pod is on
kubectl get pod <pod-name> -o wide  # shows which node
aws ec2 describe-instances --filters Name=private-dns-name,Values=<node-name>

# SSM into the node
aws ssm start-session --target <instance-id>

# Now run tcpdump directly on the node
tcpdump -i any -nn port 8080
```

Option D — kubectl sniff Plugin (Simplest Method)

```
# Install krew first, then sniff
kubectl krew install sniff

# Capture traffic from a specific pod – that's it!
kubectl sniff <pod-name> -n <namespace> -p

# Filter by port
kubectl sniff <pod-name> -n <namespace> -f 'port 8080'
```

■ **kubectl sniff** is the easiest method — it automatically handles the namespace magic, opens Wireshark locally, and gives you a GUI to browse captured traffic. Install it once and use it forever.

Real-World EKS Scenarios

■ Pod Can't Reach Another Service

```
tcpdump -i any -nn port 53                # Check DNS first
tcpdump -i any -nn host 10.0.1.45 and port 8080 # Check TCP connection
```

■ HTTP Requests Failing

```
tcpdump -i any -nn -A -s 0 port 8080
tcpdump -i any -nn -A -s 0 'tcp port 80' | grep -E 'GET|POST|HTTP|Host:'
```

■ Istio/Service Mesh Sidecar Issues

```
tcpdump -i lo port 8080 -nn -X          # Before sidecar (loopback)
tcpdump -i eth0 port 15001 -nn          # After sidecar (Envoy outbound)
```

■ ALB/NLB Health Check Debugging

```
tcpdump -i eth0 -nn 'tcp[tcpflags] & tcp-syn != 0' and port 80
```

■ Capture and Save for Wireshark

```
tcpdump -i any -nn -s 0 -w /tmp/capture.pcap -c 10000 port 8080
kubectl cp debug-pod:/tmp/capture.pcap ./capture.pcap
```

Chapter 9 — How Debug Pod Sees Pod-to-Pod Traffic

This is one of the most confusing aspects for beginners. By default, each pod lives in its own isolated network namespace. A debug pod cannot see another pod's traffic any more than you can read your neighbour's mail. Two techniques break through this isolation.

The Problem — Isolated Network Namespaces

```
EKS NODE
+-----+
|
| Pod A      Pod B      Debug Pod (normal)
| +-----+  +-----+  +-----+
| |          |---->|      | ??? | Cannot see this!
| +-----+  +-----+  +-----+
| |          |          |          |
| veth1      veth2      veth3
| +-----+-----+
|
|                NODE BRIDGE
+-----+
```

Solution 1 — `hostNetwork: true`

When you set `hostNetwork: true`, the debug pod stops being a separate network entity. It joins the node's own network namespace — it IS the node as far as networking is concerned. From here, `tcpdump` sees ALL traffic flowing through that node.

```
EKS NODE
+-----+
|
| Pod A      Pod B
| +-----+  +-----+
| |          |---->|
| +-----+  +-----+
| |          |
| veth1      veth2
| +-----+
|
|      NODE eth0 / bridge
|      ^
|
|      Debug Pod (hostNetwork:true)
|      = IS the node network = sees ALL traffic on node
+-----+
```

```
# Capture all traffic between Pod A and Pod B
tcpdump -i any -nn host 10.0.1.10 and host 10.0.4.22

# Output – you now see their traffic!
14:23:01.100 IP 10.0.1.10.54231 > 10.0.4.22.8080: Flags [P.] # A sends
14:23:01.145 IP 10.0.4.22.8080 > 10.0.1.10.54231: Flags [P.] # B replies
# Latency = 45ms
```

Solution 2 — nsenter (Enter Pod's Own Namespace)

nsenter is even more surgical — it puts you literally inside Pod A's network namespace. You see exactly what Pod A sees: its own eth0, its own routing table, its own traffic.

```
# Step 1: Find which node Pod A is on
kubectl get pod pod-a -o wide

# Step 2: SSM into that node
aws ssm start-session --target <node-instance-id>

# Step 3: Find Pod A's container ID
crictl ps | grep pod-a

# Step 4: Get the container's PID
crictl inspect <container-id> | grep pid
# Output: 'pid': 12345

# Step 5: Enter Pod A's network namespace and run tcpdump
nsenter -t 12345 -n tcpdump -i eth0 -nn -A -s 0
```

Output — from inside Pod A's perspective:

```
14:23:01.100 IP 10.0.1.10.54231 > 10.0.4.22.8080: Flags [P.] # A sending
14:23:01.145 IP 10.0.4.22.8080 > 10.0.1.10.54231: Flags [P.] # B replying
# App latency = exactly 45ms from Pod A's perspective
```

Method	What You See	Best For
Normal debug pod	Only its own traffic	Nothing useful for pod debugging
hostNetwork: true	ALL pods on that node	Quick broad capture, see everything
nsenter into Pod A	Exactly what Pod A sees	Precise per-pod debugging

■ **Mental Model:** Node = apartment building. Each Pod = separate apartment with locked door. Normal debug pod = new tenant (sees only own apartment). hostNetwork = building security guard in lobby (sees everyone). nsenter = copy of Pod A's key (you step inside their apartment).

Chapter 10 — Measuring Latency with Tcpcdump

Tcpcdump stamps every captured packet with a high-precision timestamp. By subtracting timestamps between related packets, you can calculate latency at each layer of the stack — without modifying your application code at all.

Type 1 — Network RTT (Pure Network Latency)

Measure time between SYN and SYN-ACK = pure network round-trip time.

```
tcpcdump -i any -nn 'tcp[tcpflags] & tcp-syn != 0' and host 10.0.1.45
```

```
14:23:01.100000 IP 10.0.2.30 > 10.0.1.45.8080: Flags [S]    # SYN sent
14:23:01.102500 IP 10.0.1.45 > 10.0.2.30:      Flags [S.]   # SYN-ACK received

Network RTT = 102500 - 100000 = 2.5ms    <-- healthy same-VPC latency
```

Type 2 — App Response Latency

Measure time between HTTP request and HTTP response = how long your app takes to process.

```
tcpcdump -i any -nn -A host 10.0.1.45 and port 8080
```

```
# Healthy
14:23:01.100000  POST /api/orders HTTP/1.1    # Request received
14:23:01.145000  HTTP/1.1 200 OK                          # Response sent
# App latency = 45ms ■

# Slow / broken
14:23:01.100000  POST /api/orders HTTP/1.1    # Request received
14:23:03.800000  HTTP/1.1 200 OK                          # Response finally sent
# App latency = 2.7 SECONDS  -- something is very wrong in the app
```

Type 3 — DNS Latency (Often the Hidden Killer)

```
tcpcdump -i any -nn port 53
```

```
# Good
14:23:01.100    A? orders-svc.default.svc.cluster.local
14:23:01.102    A orders-svc... 10.0.4.22    # 2ms ■

# Bad -- CoreDNS overloaded
14:23:01.100    A? orders-svc...
14:23:01.850    A orders-svc... 10.0.4.22    # 750ms  every request pays this!
```

Auto-Calculate Latency with AWK

```

tcpdump -i any -nn -tt host 10.0.1.45 and port 8080 | awk '
/Flags \[P\]/ && /GET|POST/ {req_time = $1}
/Flags \[P\]/ && /HTTP\1/ {
    if (req_time > 0)
        printf "Latency: %.3f ms\n", ($1 - req_time) * 1000
}'

```

```

Latency: 12.453 ms
Latency: 8.102 ms
Latency: 245.800 ms  <-- spike! investigate this request
Latency: 11.230 ms

```

tshark — Best for Per-Request Latency Table

```

apt-get install -y tshark

tshark -i any -nn \
  -Y "http.request or http.response" \
  -T fields \
  -e frame.time_relative \
  -e ip.src \
  -e ip.dst \
  -e http.request.method \
  -e http.request.uri \
  -e http.response.code \
  -e http.time

```

TIME	SRC	DST	METHOD	URI	CODE	LATENCY
1.000000	10.0.2.30	10.0.1.45	GET	/api/orders	-	-
1.012000	10.0.1.45	10.0.2.30	-	-	200	0.012s ok
1.500000	10.0.2.30	10.0.1.45	POST	/api/payment	-	-
3.920000	10.0.1.45	10.0.2.30	-	-	500	2.420s SLOW + ERROR

Latency Type	Healthy	Investigate	Problem
Network RTT (same VPC)	< 1ms	1–10ms	> 10ms
Network RTT (cross-AZ)	< 5ms	5–20ms	> 20ms
DNS resolution	< 5ms	5–50ms	> 50ms
HTTP app response	< 100ms	100–500ms	> 500ms
DB query (RDS same AZ)	< 5ms	5–20ms	> 20ms

Chapter 11 — Filtering by Request ID / Body

In microservices, every request typically carries a unique request ID in its JSON body or headers. Tcpdump can filter down to traffic containing a specific request ID, letting you trace a single request's journey through your entire system.

Method 1 — tcpdump + grep (Simplest)

```
# Capture full packets and grep for your request ID
tcpdump -i any -nn -A -s 0 port 8080 | grep --line-buffered -A 20 'req-id-abc123'

# IMPORTANT: --line-buffered is required for real-time output
# Without it, grep buffers output and you see nothing until the buffer fills
```

Output:

```
14:23:01.100 IP 10.0.2.30 > 10.0.1.45.8080: Flags [P.]
POST /api/orders HTTP/1.1
Content-Type: application/json
X-Request-ID: req-id-abc123

{"requestId": "req-id-abc123", "userId": "u-999", "amount": 250.00}
14:23:01.145 IP 10.0.1.45 > 10.0.2.30: Flags [P.]
HTTP/1.1 200 OK
X-Request-ID: req-id-abc123

Request at: 14:23:01.100
Response at: 14:23:01.145
Latency:    45ms
```

Method 2 — Precise Timestamps for Latency Math

```
tcpdump -i any -nn -A -s 0 -tt port 8080 \
| grep --line-buffered -E 'req-id-abc123|HTTP/1|POST|GET'
```

```
1714650181.100123 POST /api/orders HTTP/1.1
1714650181.100456 'requestId': 'req-id-abc123' # request in
1714650181.145789 HTTP/1.1 200 OK
1714650181.146001 'requestId': 'req-id-abc123' # response out
Latency = 1.145789 - 1.100123 = 45.666ms
```

Method 3 — Capture File + Analyze After (Safest in Production)

```
# Step 1: Capture to file (safe – no ASCII conversion overhead)
tcpdump -i any -nn -s 0 -w /tmp/capture.pcap -G 60 port 8080

# Step 2: Search the file for your request ID
tcpdump -r /tmp/capture.pcap -A | grep --line-buffered -A 20 'req-id-abc123'

# Step 3: Or use tshark for structured output
tshark -r /tmp/capture.pcap -Y 'http contains "req-id-abc123"'

# Step 4: Copy file to local machine for Wireshark GUI analysis
kubectl cp debug-pod:/tmp/capture.pcap ./capture.pcap
```

Method 4 — Watch Multiple Request IDs Simultaneously

```
tcpdump -i any -nn -A -s 0 port 8080 \
  | grep --line-buffered -E 'req-id-abc123|req-id-xyz789|req-id-def456'
```

```
14:23:01.100 req-id-abc123 # request 1 in
14:23:01.110 req-id-xyz789 # request 2 in
14:23:01.130 req-id-def456 # request 3 in
14:23:01.145 req-id-abc123 # request 1 out = 45ms ok
14:23:01.160 req-id-xyz789 # request 2 out = 50ms ok
14:23:01.400 req-id-def456 # request 3 out = 270ms SLOW
```

HTTPS / TLS Limitation

Protocol	Body Visible?	Solution
HTTP (port 80, 8080)	Yes — grep works perfectly	Nothing needed
HTTPS (port 443)	No — encrypted binary	Capture on loopback before encryption
Istio mTLS	No — encrypted by sidecar	Use Envoy access logs

```
# Capture BEFORE TLS encryption (loopback inside pod)
nsenter -t <pod-pid> -n tcpdump -i lo -nn -A -s 0 port 8080

# Or use Istio/Envoy access logs (already decrypted metadata)
kubectl logs <pod> -c istio-proxy | grep 'req-id-abc123'
```

Chapter 12 — Difference Between -A and -s 0

These two flags are completely different but are almost always used together. People often confuse them because they both seem related to 'seeing packet content'.

■ **Analogy:** -s 0 = bigger fishing net (catches the whole fish). -A = reading glasses (lets you read what is written on the fish). You need BOTH to catch and read a complete packet.

-s 0 — Snapshot Length (How Much to Capture)

By default, tcpdump only captures the first **96 bytes** of each packet — just enough for IP and TCP headers. The rest of the packet (including your HTTP body) is silently thrown away and is gone forever. The -s 0 flag removes this size limit.

```
Full Packet (500 bytes total)
+-----+-----+-----+-----+
| IP Header| TCP Header| HTTP Headers | HTTP Body      |
| 20 bytes| 20 bytes | 56 bytes   | 404 bytes      |
+-----+-----+-----+-----+
                        ^
      Default tcpdump cuts here (96 bytes) -- body is GONE

With -s 0:
+-----+-----+-----+-----+
| IP Header| TCP Header| HTTP Headers | HTTP Body      |
+-----+-----+-----+-----+
                        ^

Captures entire packet -- body is fully available
```

■ **Note:** -s 0 is completely silent — it does NOT change what you see on screen. It only changes how much data is stored in memory (or in a .pcap file).

-A — ASCII Display Mode (How to Show Content)

Without -A, tcpdump only prints the packet header line (timestamp, IPs, ports, flags). The -A flag tells tcpdump to also print the packet content as human-readable ASCII text.

```
# Without -A — only sees metadata
```

```
14:23:01.100 IP 10.0.2.30.54312 > 10.0.1.45.8080: Flags [P.], length 120
14:23:01.145 IP 10.0.1.45.8080 > 10.0.2.30.54312: Flags [P.], length 540
# Packets arrived -- but what is inside? No idea.
```

```
# With -A but without -s 0 -- can read but body is cut off!
```

```
POST /api/orders HTTP/1.1
Host: orders-service:8080
Content-Type: application/json

{"requestId": "abc123", "user    <-- CUT OFF HERE due to 96 byte limit
```

```
# With BOTH -A and -s 0 -- full content shown
```

```
POST /api/orders HTTP/1.1
Host: orders-service:8080
Content-Type: application/json
Content-Length: 87

{"requestId": "abc123", "userId": "u-999", "amount": 250.00} <-- COMPLETE
```

Command	Result
-A only	Content shown on screen but body is cut off at 96 bytes
-s 0 only	Full packet in memory but nothing extra shown on screen
-A -s 0 together	Full packet captured AND fully shown as readable text
Neither flag	Only packet header line shown (timestamp, IPs, flags)

When to Use What

```
# Live debugging -- read HTTP on screen
tcpdump -i any -nn -A -s 0 port 8080

# Production capture -- write to file (binary is cheaper than ASCII printing)
tcpdump -i any -nn -s 0 -w capture.pcap port 8080

# Quick connectivity check (no need to see content)
tcpdump -i any -nn port 8080
```


Chapter 13 — Can You Rewind Traffic?

■ **Short answer: NO.** Tcpcap only captures from the moment you run it. If it was not recording, that traffic is gone forever — like a CCTV camera that was not turned on.

```
YESTERDAY      2 HRS AGO      NOW
-----> time
|              |              |
[GONE]          [GONE]          You run tcpcap
Packets         Packets         [Captures from HERE only]
gone forever    gone forever
```

YES — If You Plan Ahead: Rolling Capture

```
# Always-on background capture with automatic file rotation
# -G 3600 = new file every 1 hour
# -C 100  = max 100MB per file
# -W 24   = keep only last 24 files (24 hours of history)
tcpcap -i any -nn -s 0 \
  -G 3600 \
  -C 100 \
  -W 24 \
  -w /tmp/capture_%Y%m%d_%H%M%S.pcap \
  port 8080
```

This creates a rolling 24-hour window of capture files:

```
/tmp/capture_20240501_000000.pcap <-- midnight to 1am
/tmp/capture_20240501_010000.pcap <-- 1am to 2am
...
/tmp/capture_20240501_140000.pcap <-- 2pm to 3pm (current)
# Oldest files auto-deleted to stay within -W limit
```

```
# Rewind to 2 hours ago and search for a specific request
tcpcap -r /tmp/capture_20240501_120000.pcap -A | grep -A 15 'req-id-abc123'
```

Alternative Historical Sources

Source	How Far Back	Has Body Content?	Setup Needed
Live tcpcap	Right now only	Yes	None
Rolling tcpcap files	Up to N hours (configurable)	Yes	Pre-setup required

Istio/Envoy access logs	Log retention period	No (metadata only)	Service mesh
AWS VPC Flow Logs	Weeks (configurable)	No (metadata only)	AWS config
Application logs	Log retention period	Yes (if app logs it)	App instrumentation

```
# Istio/Envoy logs -- great for request history
kubectl logs <pod> -c istio-proxy | grep 'req-id-abc123'
# Output: [2024-05-01T12:23:01] 'POST /api/orders' 200 45ms

# App logs -- if your app logs request IDs
kubectl logs <pod-name> --since=2h | grep 'req-id-abc123'
kubectl logs <pod-name> --previous | grep 'req-id-abc123' # after restart
```

Chapter 14 — Performance Impact on Pods

Tcpdump Never Adds Latency to Requests

This is the most important thing to understand: tcpdump is completely passive. The Linux kernel sends it a *copy* of each packet. Your actual packets continue their journey completely unaffected. Tcpdump is never in the traffic path.

```
WITHOUT tcpdump:
Pod A -----> Pod B
    [actual packet travels at full speed]

WITH tcpdump:
Pod A -----> Pod B
    |
    +---> Kernel sends a COPY to tcpdump process
        [original packet completely unaffected]
        [tcpdump can never slow down the original]
```

But CPU and Memory Impact is Real

Command	CPU Impact	Memory	Safe in Prod?
tcpdump -nn port 8080 (basic)	~0.5–1%	~10MB	Yes
tcpdump -nn -A port 8080	~3%	~10MB	Yes
tcpdump -nn -A -s 0 port 8080	~5–8%	~50MB	With care
tcpdump -A -s 0 (NO FILTER!)	~15–30%	~200MB+	Never

Why -A -s 0 is Heavier:

- -s 0 captures full packets — more data buffered in memory per packet
- -A converts binary to ASCII for every packet — CPU doing text conversion
- High traffic (1000 req/sec) × full packet size × ASCII conversion = CPU spike
- No filter means ALL traffic on the node is processed, not just your service

Chapter 15 — Production Safety Rules

The Golden Rules

```
# ALWAYS use specific filters -- never capture everything
tcpdump -i any -nn -A -s 0 port 8080 and host 10.0.1.10

# ALWAYS set a packet count limit
tcpdump -i any -nn -A -s 0 -c 1000 port 8080
# Stops automatically after 1000 packets

# ALWAYS write to file in production (binary cheaper than printing ASCII)
tcpdump -i any -nn -s 0 -w /tmp/cap.pcap port 8080

# Add time limit using timeout command
timeout 30 tcpdump -i any -nn -s 0 port 8080

# NEVER run without filters on high-traffic nodes
tcpdump -i any -A -s 0 # DANGEROUS -- CPU spike guaranteed

# NEVER forget -nn in EKS
tcpdump port 8080 # Generates extra DNS traffic, pollutes capture
```

Situation	Recommended Command
Quick connectivity check	<code>tcpdump -i any -nn -c 100 port 8080</code>
Debug specific pod	<code>tcpdump -i any -nn host 10.0.1.10 -c 500</code>
Inspect HTTP body	<code>tcpdump -i any -nn -A -s 0 -c 200 port 8080</code>
Filter by request ID	<code>tcpdump -i any -nn -A -s 0 port 8080 grep --line-buffered 'req-id'</code>
Long-term capture	<code>tcpdump -i any -nn -s 0 -w /tmp/cap.pcap -G 3600 -W 24 port 8080</code>
Production incident	Capture to file first with <code>-w</code> , analyze with <code>-r</code> after

When to Use Tcpdump vs Other Tools

Situation	Use Tcpdump?	Better Alternative
Pod can't reach a service	Yes	-
DNS resolution failing	Yes	-
Unexpected connection resets	Yes	-
Latency between services	Yes	tshark for automation
Security audit / traffic inspection	Yes	-
App-level bug (logic error in code)	No	App logs, APM traces

High CPU in pod	No	top, perf, profiler
Memory leak	No	kubecttl top, profiler
Historical traffic (yesterday)	No	VPC Flow Logs, app logs

Chapter 16 — Quick Reference Cheatsheet

Most Used Commands (Copy-Paste Ready)

Basic capture — see all traffic on port 8080

```
tcpdump -i any -nn port 8080
```

See packet content (HTTP bodies)

```
tcpdump -i any -nn -A -s 0 port 8080
```

Filter by specific pod and port

```
tcpdump -i any -nn host 10.0.1.45 and port 8080
```

Debug DNS issues

```
tcpdump -i any -nn port 53
```

Find specific request ID in traffic

```
tcpdump -i any -nn -A -s 0 port 8080 | grep --line-buffered -A 20 'your-request-id'
```

Capture to file for Wireshark

```
tcpdump -i any -nn -s 0 -w /tmp/capture.pcap port 8080
```

Read from saved file

```
tcpdump -r /tmp/capture.pcap -A | grep 'your-request-id'
```

New connections only (SYN)

```
tcpdump -i any -nn 'tcp[tcpflags] & tcp-syn != 0'
```

Connection resets (RST)

```
tcpdump -i any -nn 'tcp[tcpflags] & tcp-rst != 0'
```

Rolling 24hr capture

```
tcpdump -i any -nn -s 0 -G 3600 -W 24 -w /tmp/cap_%Y%m%d_%H%M.pcap port 8080
```

Auto-stop after 1000 packets

```
tcpdump -i any -nn -c 1000 port 8080
```

Traffic between two specific pods

```
tcpdump -i any -nn host 10.0.1.10 and host 10.0.4.22
```

From one pod to DB only

```
tcpdump src 10.0.1.10 and dst port 5432
```

HTTPS connectivity check

```
tcpdump -i any -nn host 10.0.1.10 and port 443
```

Watch ICMP (ping level reachability)

```
tcpdump icmp and host 10.0.1.10
```

Output Pattern Recognition

What You See	What it Means	Severity
SYN with no SYN-ACK	Server unreachable	CRITICAL
Repeated SYNs (retries)	Connection timeout	CRITICAL
RST packets	Connection rejected or killed	CRITICAL
DNS queries with no reply	CoreDNS down	CRITICAL
NXDOMAIN in DNS	Service name typo or missing	HIGH
HTTP 5xx responses	Application erroring	HIGH
DNS latency > 50ms	CoreDNS overloaded	HIGH
HTTP 4xx responses	Bad request or not found	MEDIUM
One-way packets only	Asymmetric routing issue	MEDIUM
SYN + SYN-ACK + ACK	Healthy TCP handshake	GOOD
DNS query + reply in < 5ms	CoreDNS healthy	GOOD
HTTP request + 200 response	Everything working	GOOD

Chapter 17 — Troubleshooting Decision Tree

```
Pod A cannot reach Pod B?
|
+-- Step 1: Can Pod A ping Pod B?
|   tcpdump icmp and host <pod-b-ip>
|   |
|   +-- NO ping reply --> NetworkPolicy or SecurityGroup blocking
|       |   Fix: Check 'kubectl get networkpolicy' and AWS SecurityGroups
|       |
|       +-- YES ping works --> Network layer OK, go deeper
|
+-- Step 2: Is DNS resolving correctly?
|   tcpdump -i any -nn port 53
|   |
|   +-- NXDOMAIN          --> Service name typo in your code/config
|   +-- No reply          --> CoreDNS is down, check kube-system pods
|   +-- Reply > 100ms     --> CoreDNS overloaded, scale it up
|   +-- Fast reply OK     --> DNS fine, go deeper
|
+-- Step 3: Is TCP connection establishing?
|   tcpdump -i any -nn host <pod-b> and port <port>
|   |
|   +-- SYN only, no SYN-ACK --> Pod B down or wrong port
|   +-- SYN + RST           --> Port closed, app not listening
|   +-- SYN + SYN-ACK + ACK --> Connection OK, app-level issue
|
+-- Step 4: Is the HTTP response correct?
    tcpdump -i any -nn -A host <pod-b> and port <port>
    |
    +-- HTTP 5xx    --> App is crashing or throwing exceptions
    +-- HTTP 4xx    --> Wrong endpoint, auth issue, bad request
    +-- No response--> App is hanging, check thread pool / DB
    +-- HTTP 2xx    --> Everything is working! Check app logic.
```

Where to Run Tcpdump — Decision


```

Want to debug traffic between Pod A and Pod B?
|
+-- Are they on the SAME node?
| |
| +-- YES --> Run debug pod with hostNetwork:true on that node
| |           tcpdump -i any -nn host <A> and host <B>
| |
| +-- NO  --> Run on EACH node separately
|           Or use kubectl sniff on each pod
|
+-- Need Pod A's exact perspective?
    --> Use nsenter -t <pod-a-pid> -n tcpdump -i eth0

Are you on a bastion host?
--> Use bastion only to kubectl apply debug pod or SSM into node
--> Never run tcpdump on bastion expecting to see pod traffic

```

Useful Images and Tools

Tool	Purpose	Install / Use
nicolaka/netshoot	Best debug image — has tcpdump, tshark, dig, curl, etc.	docker pull nicolaka/netshoot
tshark	CLI Wireshark — per-request latency tables	apt-get install tshark
kubectl sniff	Easiest pod capture — opens Wireshark locally	kubectl krew install sniff
Wireshark	GUI for analyzing .pcap files	wireshark.org/download.html
aws-vpc-cni	EKS CNI plugin logs for network issues	kubectl logs -n kube-system aws-node-*

This documentation was generated from real EKS debugging sessions. Network debugging in Kubernetes always comes back to these fundamentals. Start with simple filters, build up to complex expressions, and always use -nn and a filter in production.